# ElGamal Public Key Encryption

Riki Hernandez

May 2026

**Abstract**

I implement and analyze the ElGamal public key encryption scheme whose security relies on the discrete logarithm problem in $\mathbb{Z}_p^*$. The Maple implementation includes key generation encryption and decryption procedures plus the baby step giant step algorithm to demonstrate why the DLP is hard. I also connect ElGamal to my MAT401 work on Polly Cracker [5] where algebraic constructions proved vulnerable to linear algebra attacks that the DLP resists.

## Contents

**1 Introduction** **3**

**2 Historical Background** **3**

**3 Mathematical Preliminaries** **3**

**4 The ElGamal Cryptosystem** **4**

**5 Implementation** **5**

**6 Text Encoding and Message Level Operations** **6**

**7 Diffie Hellman Connection** **7**

**8 Character Sets and Alphabet Adaptability** **7**

**9 DLP Difficulty and the Baby Step Giant Step Algorithm** **7**

9.1 Naive Approach . . . . . 7

9.2 Baby Step Giant Step . . . . . 8

9.3 Maple Implementation . . . . . 8

9.4 Cracking ElGamal with BSGS . . . . . 9

**10 Security Analysis** **10**

**11 Test Cases and Demonstration** **11**
11.1 Small Hand Verifiable Example . . . . . 11
11.2 Full Text Demonstration . . . . . 12
11.3 Interactive Section . . . . . 12

**12 Conclusion** **12**

# 1 Introduction

This project implements and analyzes ElGamal public key encryption whose security relies on the discrete logarithm problem (DLP) in $\mathbb{Z}_p^*$. I chose ElGamal for its direct connection to Diffie Hellman key exchange which appears on the project cipher list and because it encrypts messages directly. Pure Diffie Hellman only establishes a shared secret and doesnt encrypt on its own so ElGamal extends it into a working cryptosystem.

This also connects to my MAT401 paper on Polly Cracker [5] where I analyzed algebraic public key cryptosystems that tried to replace number theoretic approaches like ElGamal w/ schemes based on polynomial ideal membership. Those constructions turned out vulnerable to linear algebra and Gröbner basis attacks while ElGamal's DLP based security has not been broken since 1985. I return to this contrast in Sections 9 and 12.

The notebook is organized as follows: Sections 2 and 3 cover historical background and mathematical prerequisites. Section 4 gives the formal ElGamal description w/ correctness proof. Section 5 has the Maple implementation and Section 6 extends it to text messages. Section 7 makes the Diffie Hellman connection precise. Section 8 addresses character set adaptability. Section 9 analyzes why the DLP is hard and implements baby step giant step to demonstrate this concretely. Section 10 discusses security properties. Section 11 provides test cases and Section 12 concludes.

# 2 Historical Background

In the 1970s cryptographers began searching for alternatives to symmetric key systems that would let two parties communicate securely without sharing a secret in advance. Diffie and Hellman [2] introduced public key cryptography in 1976. Their key exchange relied on the difficulty of computing discrete logarithms in $\mathbb{Z}_p^*$: given a prime $p$ a generator $g$ and the value $g^x \bmod p$ recovering $x$ appears computationally infeasible.

In 1985 ElGamal [1] extended this into a public key encryption scheme (and seperately a digital signature scheme which is outside scope here). While RSA based its security on integer factorization ElGamal reduces security directly to the discrete logarithm problem.

The relationship between Diffie Hellman and ElGamal is structural not just chronological though. As shown in Section 7 ElGamal encryption decomposes into a Diffie Hellman key exchange followed by multiplicative one time pad encryption w/ the resulting shared secret.

Both factoring based (RSA) and DLP based (ElGamal) approaches have held up under sustained cryptanalysis. The best known algorithms for both run in subexponential time (the general number field sieve for factoring, index calculus for DLP in $\mathbb{Z}_p^*$) but no polynomial time classical algorithm is known for either [3, 4].

# 3 Mathematical Preliminaries

We recall the definitions and results needed for ElGamal.

**Definition 1.** *A group $G$ is* cyclic *if there exists $g \in G$ such that every element of $G$ can be written as a power of $g$. Such a $g$ is called a* generator *(or* primitive root*) of $G$.*

**Definition 2.** *The* order *of an element $a$ in a group $G$ written* $\mathrm{ord}(a)$ *is the smallest positive integer $d$ such that $a^d = e$ where $e$ is the identity. In $\mathbb{Z}_p^*$ the identity is 1 so* $\mathrm{ord}(a)$ *is the smallest $d$ w/ $a^d \equiv 1 \pmod p$.*

**Theorem 1.** *For any prime $p$ the multiplicative group $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$ under multiplication modulo $p$ is cyclic of order $p-1$.*

This is standard (Koblitz [3] Theorem 2.8). An element $g$ is a generator of $\mathbb{Z}_p^*$ iff $\mathrm{ord}(g) = p-1$. To verify $g$ is a generator it suffices to check $g^{(p-1)/q} \not\equiv 1 \pmod p$ for each prime factor $q$ of $p-1$. Since if $g^{(p-1)/q} \equiv 1$ for some prime $q \mid p-1$ then $\mathrm{ord}(g) \mid (p-1)/q < p-1$ hence $g$ is not a generator.

**Definition 3** (Discrete Logarithm Problem)**.** *Given a prime $p$ a generator $g$ of $\mathbb{Z}_p^*$ and an element $h \in \mathbb{Z}_p^*$ find the integer $x \in \{0, \ldots, p-2\}$ such that $g^x \equiv h \pmod p$.*

Computing $g^x \bmod p$ is efficient via repeated squaring: write $x$ in binary as $x = \sum_i b_i 2^i$ and compute $g^x = \prod_i (g^{2^i})^{b_i}$ squaring at each step and multiplying in when the bit is 1. This takes $O(\log x)$ multiplications mod $p$. No polynomial time algorithm is known for the inverse problem.

**Theorem 2** (Fermat's Little Theorem)**.** *For any prime $p$ and integer $a$ not divisible by $p$,*

$$a^{p-1} \equiv 1 \pmod p$$

*Consequently $a^{-1} \equiv a^{p-2} \pmod p$.*

In ElGamal decryption we need $c_1^{-x} \bmod p$. Since $c_1^{p-1} \equiv 1 \pmod p$ by fermat's little theorem it follows that

$$c_1^{p-1-x} = c_1^{p-1} \cdot c_1^{-x} \equiv 1 \cdot c_1^{-x} = c_1^{-x} \pmod p$$

Thus we avoid explicit modular inverse computation and just compute $c_1^{p-1-x} \bmod p$ via repeated squaring which gives $c_1^{-x} \bmod p$ directly.

# 4 The ElGamal Cryptosystem

The ElGamal encryption scheme [1] consists of three algorithms.

**Key Generation.** Choose a large prime $p$ and a generator $g$ of $\mathbb{Z}_p^*$. Choose random private key $x \in \{2, 3, \ldots, p-2\}$ and compute public key $h = g^x \bmod p$. Public key is $(p, g, h)$ and private key is $x$.

**Encryption.** To encrypt message $m \in \{1, \ldots, p-1\}$ choose random ephemeral key $k \in \{2, 3, \ldots, p-2\}$. Compute

$$c_1 = g^k \bmod p, \qquad c_2 = m \cdot h^k \bmod p$$

The ciphertext is the pair $(c_1, c_2)$.

**Decryption.** Given ciphertext $(c_1, c_2)$ and private key $x$ compute

$$m = c_2 \cdot c_1^{p-1-x} \bmod p$$

**Theorem 3** (Correctness)**.** *For any message $m$ encrypted under a valid ElGamal key pair decryption recovers $m$.*

*Proof.* Since $c_1 = g^k$ and $c_2 = m \cdot h^k$ w/ all computations in $\mathbb{Z}_p^*$:

$$\begin{aligned} c_2 \cdot c_1^{-x} &\equiv (m \cdot h^k)(g^k)^{-x} \pmod{p} \\ &\equiv m \cdot (g^x)^k \cdot g^{-xk} \pmod{p} \\ &\equiv m \cdot g^{xk} \cdot g^{-xk} \pmod{p} \\ &\equiv m \pmod{p} \end{aligned}$$

We compute $c_1^{-x} \bmod p$ as $c_1^{p-1-x} \bmod p$ by fermat's little theorem (Section 3): since $c_1^{p-1} \equiv 1 \pmod{p}$ it follows that $c_1^{p-1-x} \equiv c_1^{-x} \pmod{p}$. □

Each encryption uses fresh random $k$ so encrypting the same message twice produces different ciphertexts. This is important for the security analysis in Section 10.

# 5 Implementation

The Maple implementation has five core procedures and four text handling procedures. All modular arithmetic uses `Power(base, exp) mod p` for efficient modular exponentiation and `modp(expr, p)` for reduction to $[0, p-1]$.

```
GeneratePrime := proc(bits)
    local lo, hi, candidate;
    lo := 2^(bits - 1);
    hi := 2^bits;
    candidate := rand(lo .. hi)();
    return nextprime(candidate);
end proc:

FindGenerator := proc(p)
    return PrimitiveRoot(p);
end proc:

ElGamalKeyGen := proc(bits)
    local p, g, x, h;
    p := GeneratePrime(bits);
    g := FindGenerator(p);
    x := rand(2 .. p - 2)();
    h := Power(g, x) mod p;
    return [p, g, x, h];
end proc:

ElGamalEncrypt := proc(m, p, g, h)
    local k, c1, c2;
    if m < 1 or m >= p then
```

```
        error "Message m must satisfy 1 <= m <= p-1.";
    end if;
    k := rand(2 .. p - 2)();
    c1 := Power(g, k) mod p;
    c2 := modp(m * Power(h, k), p);
    return [c1, c2];
end proc:

ElGamalDecrypt := proc(c1, c2, p, x)
    local s_inv, m;
    s_inv := Power(c1, p - 1 - x) mod p;
    m := modp(c2 * s_inv, p);
    return m;
end proc:
```

# 6 Text Encoding and Message Level Operations

ElGamal encrypts integers in $\{1, \ldots, p-1\}$. To encrypt text we convert each character to its ASCII byte value and encrypt each one individually as a seperate ElGamal ciphertext pair $(c_1, c_2)$. This character by character encoding shows the probabilistic property since the same letter encrypts differently each time because of fresh random $k$. The tradeoff is 2:1 ciphertext expansion discussed further in Section 10.

```
TextToNumbers := proc(text)
    return convert(text, bytes);
end proc:

NumbersToText := proc(nums)
    local safe_nums;
    if nops(nums) = 0 then return "" end if;
    safe_nums := remove(x -> x = 0, nums);
    if nops(safe_nums) = 0 then return "" end if;
    return convert(safe_nums, bytes);
end proc:

ElGamalEncryptMessage := proc(text, p, g, h)
    local bytes_list, i;
    bytes_list := TextToNumbers(text);
    return [seq(ElGamalEncrypt(bytes_list[i], p, g, h),
                i = 1 .. nops(bytes_list))];
end proc:

ElGamalDecryptMessage := proc(cipherList, p, x)
    local i, decrypted_bytes;
    decrypted_bytes := [seq(
        ElGamalDecrypt(cipherList[i][1], cipherList[i][2], p, x),
```

```
        i = 1 .. nops(cipherList))];
    return NumbersToText(decrypted_bytes);
end proc:
```

# 7 Diffie Hellman Connection

ElGamal encryption decomposes into a Diffie Hellman key exchange followed by multiplicative one time pad encryption. In Diffie Hellman Alice holds private key $x$ and publishes $h = g^x \bmod p$ while Bob chooses ephemeral $k$ and sends $g^k \bmod p$. Both can compute the shared secret $s = g^{xk} \bmod p$ (Alice computes $(g^k)^x$ and Bob computes $h^k$).

In ElGamal $c_1 = g^k \bmod p$ is exactly Bob's Diffie Hellman public value and $c_2 = m \cdot h^k \bmod p$ masks the message w/ the shared secret $h^k = g^{xk}$. Decryption recovers $s = c_1^x = g^{xk} \bmod p$ and unmasks via $m = c_2 \cdot s^{-1} \bmod p$. Since each encryption uses fresh $k$ the one time pad is genuinely one time: every message gets masked by its own shared secret.

# 8 Character Sets and Alphabet Adaptability

ElGamal operates over $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$ for any prime $p$. The "alphabet" of the cipher is this entire set. Any character encoding maps into ElGamal's domain as long as $p$ exceeds the largest codepoint value. For ASCII (7 bit) any prime $p > 127$ suffices. For extended ASCII (8 bit) $p > 255$. For full Unicode (UTF-32) any prime exceeding 1,114,111 works. The prime $p$ gets chosen to accommodate any character set without modifying the algorithm.

Compare this w/ other ciphers on the project list. The Solitaire cipher is tied to a standard deck of 52 cards plus 2 jokers and naturally operates on 26 letters so extending it to larger character sets would require redesigning the keystream generator. The Four Square cipher requires a $5 \times 5$ grid w/ I and J merged. ElGamal has no structural constraint on alphabet size: $p$ just needs to exceed the largest character value.

Our implementation uses ASCII encoding (byte values 32-126 for printable characters). W/ a 32 bit or larger prime there is ample room.

# 9 DLP Difficulty and the Baby Step Giant Step Algorithm

ElGamal's security rests on the assumption that the discrete logarithm problem is computationally hard. Here we make this concrete by implementing the best known generic attack on the DLP and demonstrating its limitations.

## 9.1 Naive Approach

The simplest attack on the DLP is exhaustive search: given $g$ $h$ and $p$ try $x = 0, 1, 2, \ldots$ until $g^x \equiv h \pmod p$. This requires $O(p)$ multiplications in the worst case. For a 64 bit prime

this means up to $2^{64} \approx 1.8 \times 10^{19}$ operations well beyond practical computation. Figure 1 shows how quickly exhaustive search becomes impractical relative to BSGS.

## 9.2 Baby Step Giant Step

Shanks' baby step giant step (BSGS) algorithm [3] improves on exhaustive search via a time space tradeoff. Write $x = im + j$ where $m = \lceil \sqrt{p-1} \rceil$ and $0 \leq j < m$. Then $g^x = g^{im+j}$ hence $h \cdot g^{-im} = g^j$. The algorithm has two phases:

*Baby step.* Compute and store $g^j \bmod p$ for $j = 0, 1, \ldots, m-1$ in a hash table.

*Giant step.* Compute $h \cdot (g^{-m})^i \bmod p$ for $i = 0, 1, \ldots, m-1$. If it matches a table entry $g^j$ then $x = im + j$.

Since $m = \lceil \sqrt{p-1} \rceil$ and $0 \leq x \leq p-2 < m^2$ the division $x = im + j$ w/ $i = \lfloor x/m \rfloor$ and $j = x \bmod m$ satisfies $0 \leq i < m$ and $0 \leq j < m$. Thus both the baby step table and giant step scan have at most $m$ entries and the algorithm is guarenteed to find $x$.

It performs $O(\sqrt{p})$ multiplications and uses $O(\sqrt{p})$ space so its a quadratic improvement over exhaustive search but still exponential in the bit length of $p$ since for a $b$-bit prime BSGS requires $O(2^{b/2})$ operations.

This is why increasing prime size is so effective. A 32 bit prime needs about $2^{16} \approx 65{,}000$ steps. A 64 bit prime about $2^{32} \approx 4 \times 10^9$. A 128 bit prime about $2^{64} \approx 10^{19}$. At 2048 bits (NIST recommendation) BSGS would need about $2^{1024}$ steps which exceeds any conceivable computation.

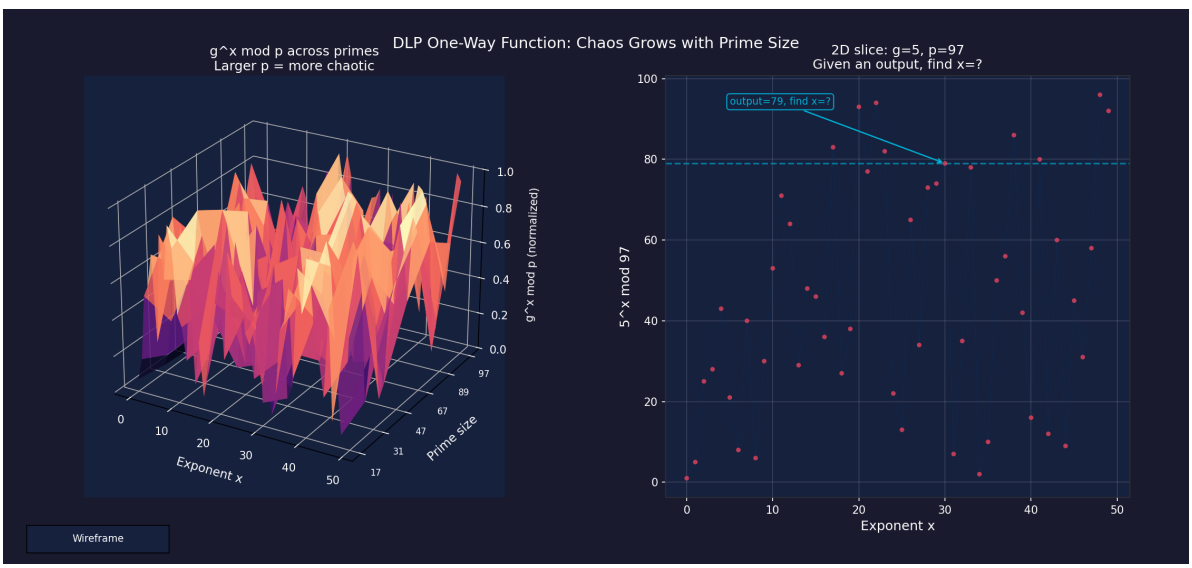


Figure 1: Left: 3D surface of $g^x \bmod p$ for several bases, showing chaotic output that makes inversion intractable. Right: 2D slice for a single generator showing the pseudorandom scatter with an inverse problem annotation.

## 9.3 Maple Implementation

```
BSGS := proc(g, h, p)
    local m, tbl, power, j, factor, gamma, i;
    m := isqrt(p - 1) + 1;
    # Baby step: build table of g^j mod p
    tbl := table();
    power := 1;
    for j from 0 to m - 1 do
        tbl[power] := j;
        power := modp(power * g, p);
    end do;
    # Giant step: check h * (g^(-m))^i against table
    factor := Power(g, p - 1 - m) mod p;
    gamma := h;
    for i from 0 to m - 1 do
        if assigned(tbl[gamma]) then
            return i * m + tbl[gamma];
        end if;
        gamma := modp(gamma * factor, p);
    end do;
    return FAIL;
end proc:
```

## 9.4 Cracking ElGamal with BSGS

We use BSGS to break our own ElGamal keys at various sizes. On a 16 bit prime ($p \approx 2^{16}$) the table has about $\sqrt{p} \approx 256$ entries and BSGS finishes instantly. At 24 bits the table grows to about 4,096 entries. At 32 bits ($\sqrt{p} \approx 65{,}000$) computation is still fast but the table gets substantial. To measure scaling precisely I wrote a companion Python implementation that reimplements ElGamal and BSGS runs automated benchmarks across key sizes and generates the plots in Figures 1, 2, and 3. Maple works for interactive demonstration but Python w/ matplotlib gives finer control over timing and visualization. The results:

| **Bits** | **Avg time** | $\sqrt{p}$ |
|---|---|---|
| 12 | 0.00001 s | 64 |
| 16 | 0.00003 s | 256 |
| 20 | 0.0001 s | 1,024 |
| 24 | 0.0005 s | 4,096 |
| 28 | 0.002 s | 16,384 |
| 32 | 0.014 s | 65,536 |
| 36 | 0.053 s | 262,144 |
| 40 | 0.29 s | 1,048,576 |

Each 4 bit increase roughly quadruples solve time confirming $O(2^{b/2})$ scaling. Extrapolating to production sizes: at 2048 bits BSGS would need approximately $2^{1024}$ table entries which exceeds the number of atoms in the observable universe ($\approx 10^{80}$) by a factor that itself exceeds the number of atoms in the observable universe.

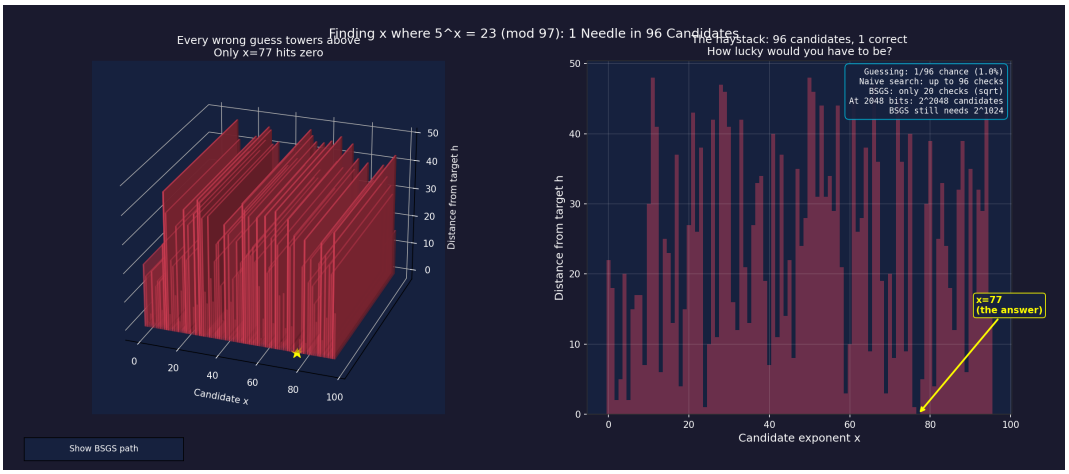


Figure 2: Needle in a haystack: each bar represents a candidate exponent $x \in \{0, \ldots, p-2\}$ with height equal to $|g^x \bmod p - h|$. Only the correct $x$ has height zero. Guessing correctly requires $1/(p-1)$ luck. The annotation compares naive search ($O(p)$) with BSGS ($O(\sqrt{p})$).

# 10 Security Analysis

ElGamal's security reduces to the computational Diffie Hellman (CDH) problem: given $g$, $g^x$, $g^k$ compute $g^{xk}$. CDH is believed equivalent to DLP in $\mathbb{Z}_p^*$ though this equivalence hasnt been proven in general. As demonstrated in Section 9 the best known generic algorithm (BSGS) has complexity $O(\sqrt{p})$ which is exponential in the bit length. More sophisticated algorithms like the number field sieve achieve subexponential complexity $L_p[1/3, c]$ [3, 4] but remain impractical for primes of 2048 bits or more.

Compare the DLP based approach w/ algebraic approaches to public key cryptography. In my MAT401 paper [5] I analyzed the Polly Cracker cryptosystem introduced by Fellows and Koblitz [6] which based its security on ideal membership in polynomial rings rather than the DLP. The hope was that algebraic problems drawn from NP hard combinatorial instances (graph 3 coloring, perfect codes) could serve as trapdoor one way functions. But structured instances from these graph based constructions leak enough information to make linear algebra attacks effective [5]. The ciphertexts expose a linear transformation $\mathbf{c}' = A\mathbf{c}$ and an attacker constructs a decryption vector by solving $\mathbf{w}^\top A = \mathbf{1}^\top$ over $\mathbb{F}_2$ bypassing the private key entirely. The DLP doesnt have this kind of exploitable algebraic structure.

Since each encryption uses fresh random $k$ the same plaintext produces different ciphertexts each time. This gives semantic security against chosen plaintext attacks (IND CPA) under the decisional Diffie Hellman assumption [4]. An adversary cant distinguish encryptions of two chosen messages which deterministic schemes like textbook RSA dont have. Figure 3 visualizes this: 200 encryptions of the same character scatter uniformly over $\mathbb{Z}_p^* \times \mathbb{Z}_p^*$ and ciphertexts of distinct characters overlap completely.

The cost is ciphertext expansion since each plaintext element $m$ produces a pair $(c_1, c_2)$ doubling data size. In our character by character encoding each character expands to two group elements.

ElGamal is also malleable. Given ciphertext $(c_1, c_2)$ encrypting $m$ an adversary can

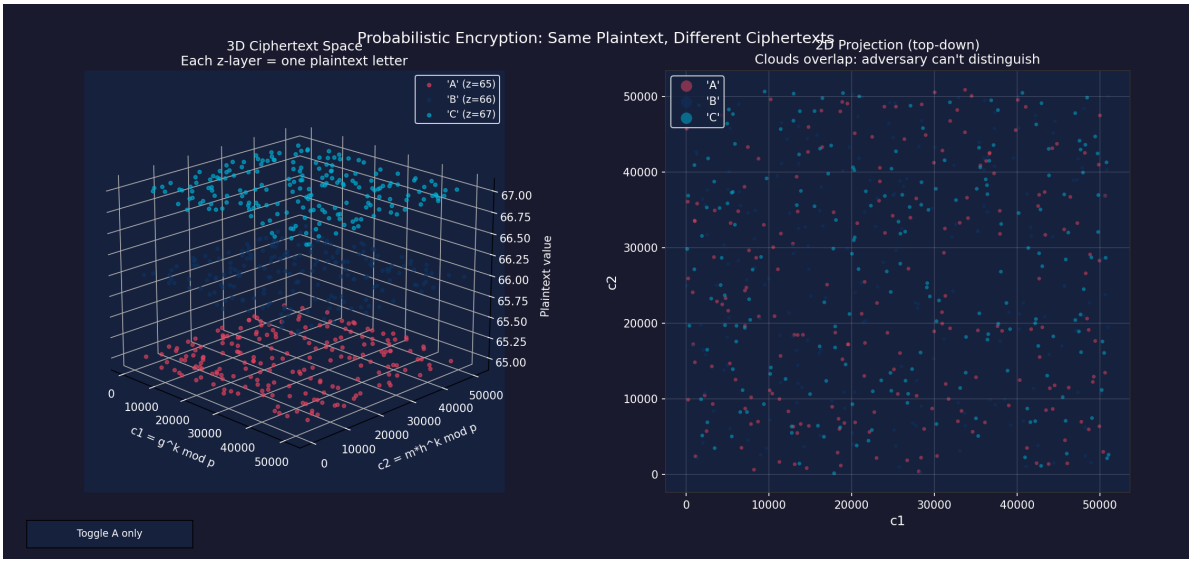


Figure 3: 3D ciphertext space: each letter (A, B, C) encrypted 200 times forms a cloud at its ASCII value on the $z$-axis, scattered uniformly in the $(c_1, c_2)$ plane. The 2D projection (right) shows these clouds overlap completely, so an adversary cannot determine which character was encrypted.

compute $(c_1, c_2 \cdot r)$ which encrypts $m \cdot r$ without knowing $m$ or the private key. So ElGamal is not secure against adaptive chosen ciphertext attacks (IND CCA2). In practice you fix this by combining ElGamal w/ a message authentication code or using Cramer Shoup which achieves CCA2 security under the DDH assumption [4].

For this educational implementation we use primes of 32 to 64 bits. Production security requires at least 2048 bits due to advances in the number field sieve.

# 11 Test Cases and Demonstration

## 11.1 Small Hand Verifiable Example

For $p = 23$ and $g = 5$. To verify 5 is a primitive root modulo 23 note $p - 1 = 22 = 2 \cdot 11$. By the criterion from Section 3 we check the two prime factors:

$$5^{22/2} = 5^{11} \equiv 22 \equiv -1 \not\equiv 1 \pmod{23}, \qquad 5^{22/11} = 5^2 = 25 \equiv 2 \not\equiv 1 \pmod{23}$$

Since neither gets 1 it follows that $\mathrm{ord}(5) = 22 = p - 1$ hence 5 is a generator of $\mathbb{Z}_{23}^*$.

Let $x = 6$ so $h = g^x \bmod p = 5^6 \bmod 23 = 8$.

To encrypt $m = 7$ choose $k = 3$. Then

$$\begin{aligned} c_1 &= 5^3 \bmod 23 = 125 \bmod 23 = 10 \\ h^k \bmod p &= 8^3 \bmod 23 = 512 \bmod 23 = 6 \\ c_2 &= 7 \cdot 6 \bmod 23 = 42 \bmod 23 = 19 \end{aligned}$$

Ciphertext is $(10, 19)$.

To decrypt compute

$$c_1^{p-1-x} \bmod p = 10^{16} \bmod 23 = 4$$

then

$$m = c_2 \cdot 4 \bmod 23 = 19 \cdot 4 \bmod 23 = 76 \bmod 23 = 7$$

recovering the original message.

We can also verify BSGS on this: `BSGS(5, 8, 23)` returns $x = 6$ confirming it correctly recovers the private key from the public key.

### 11.2 Full Text Demonstration

In the Maple worksheet we generate a 64 bit key pair and encrypt "Number theory is beautiful." The ciphertext consists of 27 pairs of integers. Decryption recovers the original plaintext. Encrypting the same sentence again produces entirely different ciphertext pairs demonstrating the probabilistic property.

### 11.3 Interactive Section

The Maple worksheet includes a section where the user can edit a string variable and execute cells to encrypt and decrypt messages of their choosing. Pregenerated keys are loaded for convenience and fresh keys can be generated by calling `ElGamalKeyGen(64)`.

## 12 Conclusion

In this project I implemented ElGamal public key encryption in Maple and analyzed its security from both theoretical and computational perspectives. The implementation provides callable procedures for key generation encryption and decryption that handle arbitrary text messages and the correctness proof in Section 4 confirms decryption always recovers the original plaintext.

The baby step giant step demonstration in Section 9 makes the security argument concrete. We broke our own ElGamal keys at 16 24 and 32 bits showing BSGS succeeds quickly at small sizes but scales as $O(\sqrt{p})$ making it impractical as the prime grows. This square root barrier isnt specific to BSGS. It is the best any generic DLP algorithm can achieve [3, 4]. The number field sieve does better for specific groups like $\mathbb{Z}_p^*$ but its subexponential complexity $L_p[1/3, c]$ still requires primes of at least 2048 bits for security at current computational limits.

The DLP's resistance to algorithmic attack contrasts w/ the algebraic approaches I analyzed in my MAT401 paper [5]. The Polly Cracker constructions based security on ideal membership in polynomial rings encoding NP hard combinatorial problems (graph coloring, perfect codes) as generating sets for polynomial ideals. The hope was these would serve as trapdoor functions. But as I showed in that paper the specific algebraic structure of those constructions leaks enough information for linear algebra attacks over $\mathbb{F}_2$ to bypass the private key entirely. Basically all proposed variants of the original Polly Cracker framework have been defeated [5] while ElGamal's DLP based security remains intact after 40 years.

So a cryptosystem's security depends not only on nominal hardness of its underlying problem but on whether the construction's algebraic structure leaks information that makes the problem tractible in practice.

# References


[1] T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469-472, July 1985.

[2] W. Diffie and M. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, November 1976.

[3] N. Koblitz, *A Course in Number Theory and Cryptography*, 2nd ed. New York: Springer Verlag, 1994.

[4] A. Menezes, P. van Oorschot, and S. Vanstone, *Handbook of Applied Cryptography*, Boca Raton: CRC Press, 1996.

[5] R. Hernandez, "Polly Cracker: Concretions of A Combinatorial Algebraic Public Key Cryptosystem," MAT401 Final Paper, December 2025.

[6] N. Koblitz, *Algebraic Aspects of Cryptography*, Springer Verlag, 1998.